Solar Optimizer: Building a Predictive Maintenance Script for Peak Photovoltaic Efficiency.

Final Report

Completed on
Aug 29, 2026

Prepared by
Kashyap K

Introduction

Solar panels are electronic devices that absorb sunlight and convert it directly into usable electricity. This is a common source of renewable energy. The energy production is clean and does not produce greenhouse gases. However, it is important to ensure that the solar panels are arranged at the right angle and location to maximize the amount of sunlight it receives and improve the efficiency of energy production. Wrong angle, cloud cover that blocks sunlight or soiling of the panels could significantly lower the efficiency.

A predictive model to initiate maintenance activity on large solar farms is essential to reduce power interruption while controlling the cost involved in performing frequent repair activity. The project attempts to build such a predictive model based on a correlation analysis of data available from solar plants.

The typical parameters affecting power generation include:

- **Solar Irradiance**: A direct measure of light energy hitting the panels. The higher the sunlight, the more electricity that can be generated.
- **Ambient temperature:** The temperature of the air around the solar panels.
- **Module temperature**: The actual temperature of the solar panel itself.
- **Panel Angle:** The angle at which the solar panel is positioned toward the sun.

- **Soiling index**: A measure of how much dirt or debris has been built up upon the solar panel. A higher soiling index means more dirt blocking the sunlight, consequently causing power output to drop.

Methodology

Description of approach

The initial approach attempts to derive the correlation between different variables that affect power generation.

https://www.kaggle.com/datasets/anikannal/solar-power-generation-data has relevant datasets that capture power generation and sensor data. “This data has been gathered at two solar power plants in India over a 34 day period. It has two pairs of files - each pair has one power generation dataset and one sensor readings dataset. The power generation datasets are gathered at the inverter level - each inverter has multiple lines of solar panels attached to it. The sensor data is

gathered at a plant level - a single array of sensors optimally placed at the plant."

In order to explore the correlation between the sensor data and power generation, python scripts on Google Colab were developed. The first step involved downloading the above data sets and loading it into a data frame for further analysis.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Define file paths
plant1_generation_path = '/content/Plant_1_Generation_Data.csv'
plant1_weather_path = '/content/Plant_1_Weather_Sensor_Data.csv'


# --- Process Plant 1 Data ---
print("\n--- Processing Plant 1 ---")
try:
  df_gen1 = pd.read_csv(plant1_generation_path)
  df_weather1 = pd.read_csv(plant1_weather_path)
except FileNotFoundError as e:
  print(f"Error loading Plant 1 data: {e}")
  df_gen1 = pd.DataFrame()
  df_weather1 = pd.DataFrame()

if not df_gen1.empty and not df_weather1.empty:
  # Convert DATE_TIME to datetime objects
  df_gen1['DATE_TIME'] = pd.to_datetime(df_gen1['DATE_TIME'], errors='coerce', dayfirst=True)
  df_weather1['DATE_TIME'] = pd.to_datetime(df_weather1['DATE_TIME'], errors='coerce')

  # Drop rows where DATE_TIME conversion failed
  df_gen1.dropna(subset=['DATE_TIME'], inplace=True)
  df_weather1.dropna(subset=['DATE_TIME'], inplace=True)

  # Merge Plant 1 data for Ambient Temp vs Total Yield
  merged_df_plant1 = pd.merge(
    df_gen1[['DATE_TIME', 'PLANT_ID', 'TOTAL_YIELD']],
    df_weather1[['DATE_TIME', 'PLANT_ID', 'AMBIENT_TEMPERATURE']],
    on=['DATE_TIME', 'PLANT_ID'],
    how='inner'
  )

  print("Plant 1 Merged DataFrame Head (Ambient Temp vs Total Yield):")
  display(merged_df_plant1.head())
```

An inspection of the first few rows for the data set indicates the relevant dimensions. For power generation, AC and DC power columns from the dataset are used. Ambient temperature, module temperature and irradiation columns are used for the sensor values that impact power generation.

First 5 rows of the dataset: Plant_1_Generation_Data.csv

Row	DATE_TIME	PLANT_ID	SOURCE_KEY	DC_POWER	AC_POWER	DAILY_YIELD	TOTAL_YIELD
0	15-05-2020 00:00	4135001	1BY6WEcLGh8j5v7	0.0	0.0	0.0	6259559.0
1	15-05-2020 00:00	4135001	1IF53ai7Xc0U56Y	0.0	0.0	0.0	6183645.0
2	15-05-2020 00:00	4135001	3PZuoBAID5Wc2HD	0.0	0.0	0.0	6987759.0
3	15-05-2020 00:00	4135001	7JYdWkrLSPkdwr4	0.0	0.0	0.0	7602960.0
4	15-05-2020 00:00	4135001	McdE0feGgRqW7Ca	0.0	0.0	0.0	7158964.0

First 5 rows of the dataset Plant_2_Weather_Sensor_Data.csv

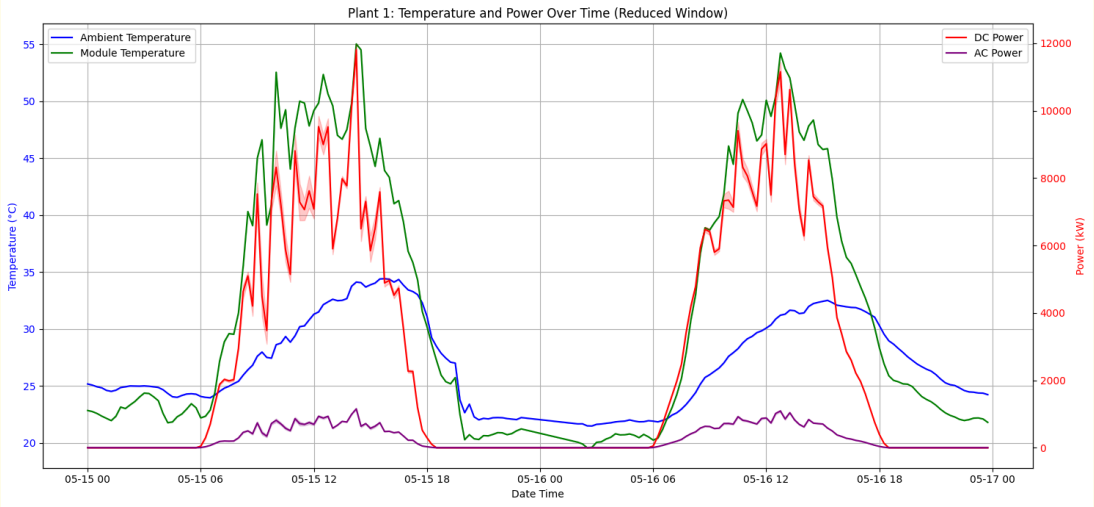
Row	DATE_TIME	PLANT_ID	SOURCE_KEY	AMBIENT_TEMPERATURE	MODULE_TEMPERATURE	IRRADIATION
0	2020-05-15 00:00:00	4136001	iq8k7ZNt4Mwm3w0	27.004764	25.060789	0.0
1	2020-05-15 00:15:00	4136001	iq8k7ZNt4Mwm3w0	26.880811	24.421869	0.0
2	2020-05-15 00:30:00	4136001	iq8k7ZNt4Mwm3w0	26.682055	24.427290	0.0
3	2020-05-15 00:45:00	4136001	iq8k7ZNt4Mwm3w0	26.500589	24.420678	0.0
4	2020-05-15 01:00:00	4136001	iq8k7ZNt4Mwm3w0	26.596148	25.088210	0.0

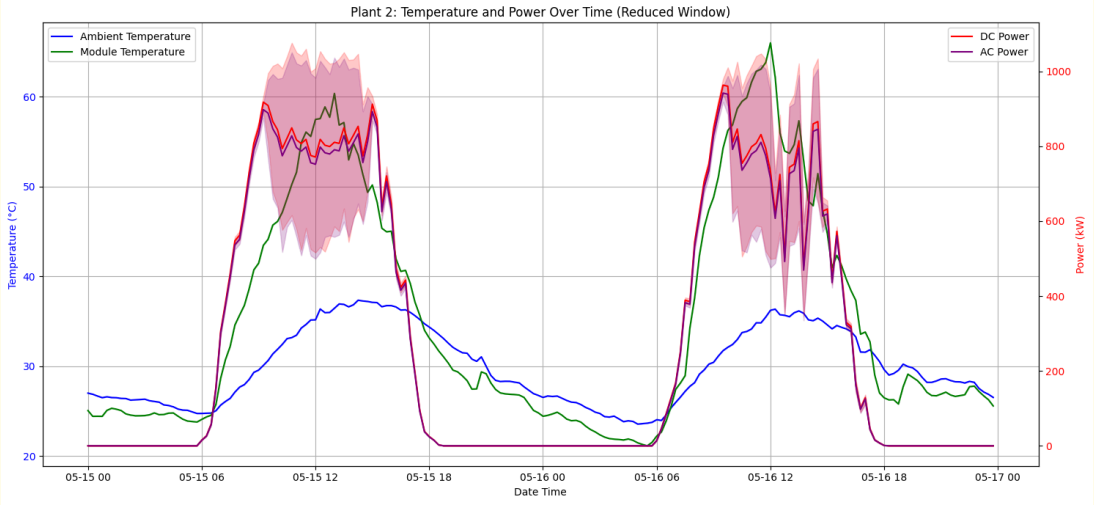
Findings

The initial step to visualize the correlation is to generate a time series data for each of the plants. To accommodate the large variance in values between each dimension, the time series is plotted against different y-axis values that help observe the correlation.

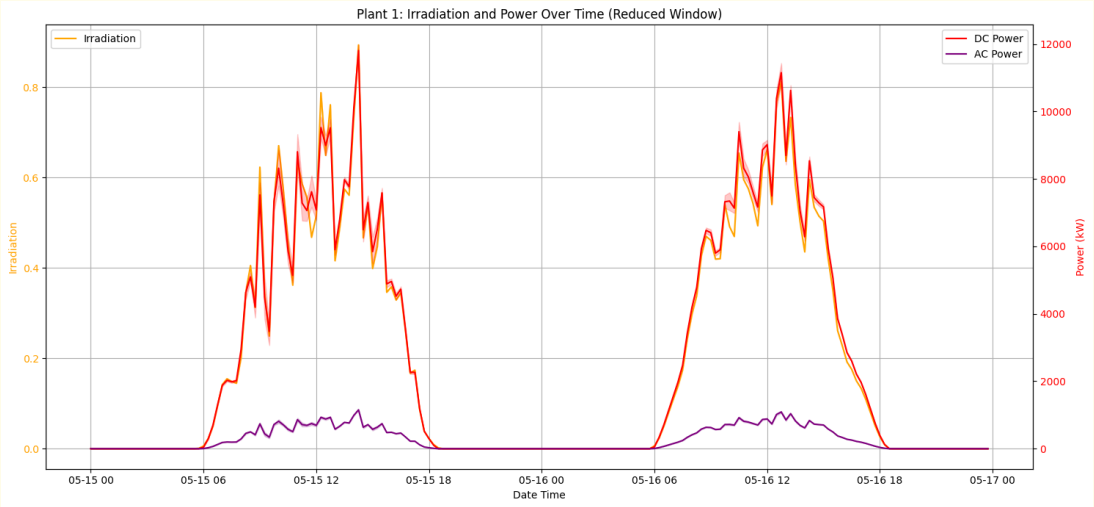
```
# --- Plot Time Series for Plant 2 (Irradiation vs Power) ---
    fig, ax1 = plt.subplots(figsize=(15, 7))

    # Plot Irradiation on the primary Y-axis (ax1)
    sns.lineplot(x='DATE_TIME', y='IRRADIATION', data=merged_power_weather_plant2, ax=ax1,
label='Irradiation', color='orange')

    ax1.set_xlabel('Date Time')
    ax1.set_ylabel('Irradiation', color='orange')
    ax1.tick_params(axis='y', labelcolor='orange')
    ax1.legend(loc='upper left')
    ax1.grid(True)

    # Create a secondary Y-axis for Power (ax2)
    ax2 = ax1.twinx()
    sns.lineplot(x='DATE_TIME', y='DC_POWER', data=merged_power_weather_plant2, ax=ax2, label='DC
Power', color='red')
    sns.lineplot(x='DATE_TIME', y='AC_POWER', data=merged_power_weather_plant2, ax=ax2, label='AC
Power', color='purple')

    ax2.set_ylabel('Power (kW)', color='red')
    ax2.tick_params(axis='y', labelcolor='red')
    ax2.legend(loc='upper right')

    plt.title('Plant 2: Irradiation and Power Over Time (Reduced Window)')
    fig.tight_layout()
    plt.show()
```

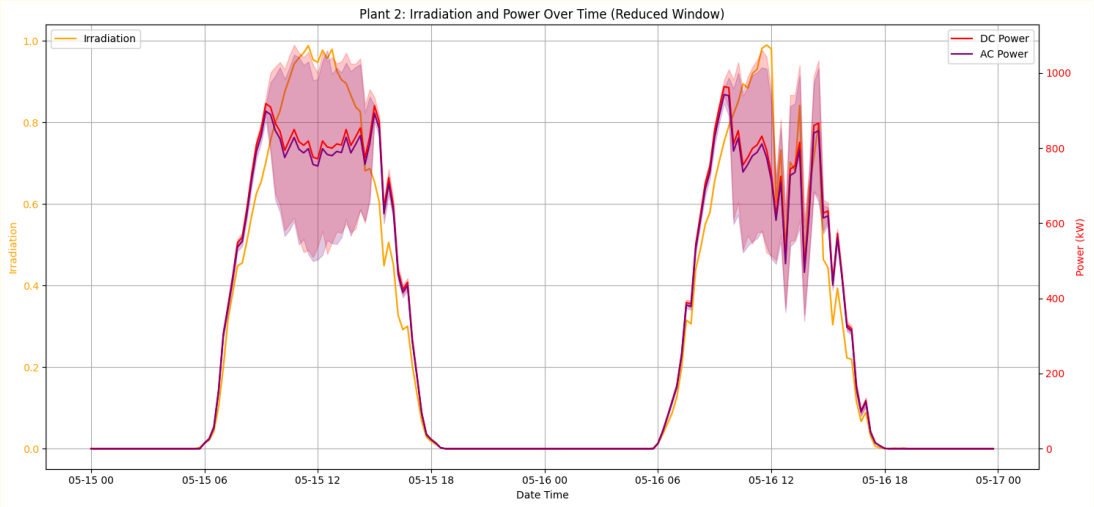
The time window had to be reduced to generate the plot quicker. The positive correlation between temperature and power is evident from these graphs for both the plants.





As expected, irradiation shows a positive correlation with power generation. This is plotted separately to improve observability across the variables.





In order to formalize the correlation, I used the correlation function in python. This computes Pearson correlation coefficient that quantifies the strength and direction of a linear relationship between two variables. The values lie between -1 and 1, where negative values indicate inverse relationship, 0 indicates no correlation and positive values indicate direct correlation.

```
correlation_matrix = df.corr(numeric_only=True)
```

The correlation is visualized using a scatter plot.

```
    # Calculate correlations for Plant 2 (Irradiation vs Power Generation)
    corr_plant2_irradiation_dc =
merged_power_weather_plant2['IRRADIATION'].corr(merged_power_weather_plant2['DC_POWER'])
    corr_plant2_irradiation_ac =
merged_power_weather_plant2['IRRADIATION'].corr(merged_power_weather_plant2['AC_POWER'])
    print(f"Correlation between Irradiation and DC Power for Plant 2: {corr_plant2_irradiation_dc:.4f}")
    print(f"Correlation between Irradiation and AC Power for Plant 2: {corr_plant2_irradiation_ac:.4f}")
```

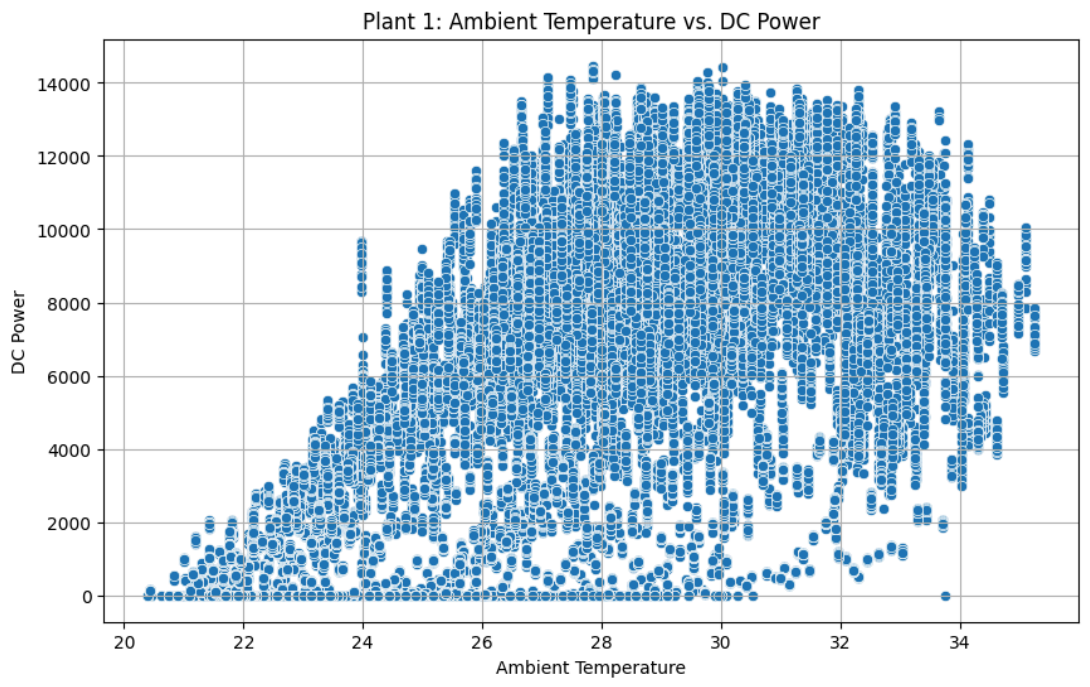
```
# Plot scatter chart for Plant 2 (Irradiation vs DC Power)
plt.figure(figsize=(10, 6))
sns.scatterplot(x='IRRADIATION', y='DC_POWER', data=merged_power_weather_plant2)
plt.title('Plant 2: Irradiation vs. DC Power')
plt.xlabel('Irradiation')
plt.ylabel('DC Power')
plt.grid(True)
plt.show()
```

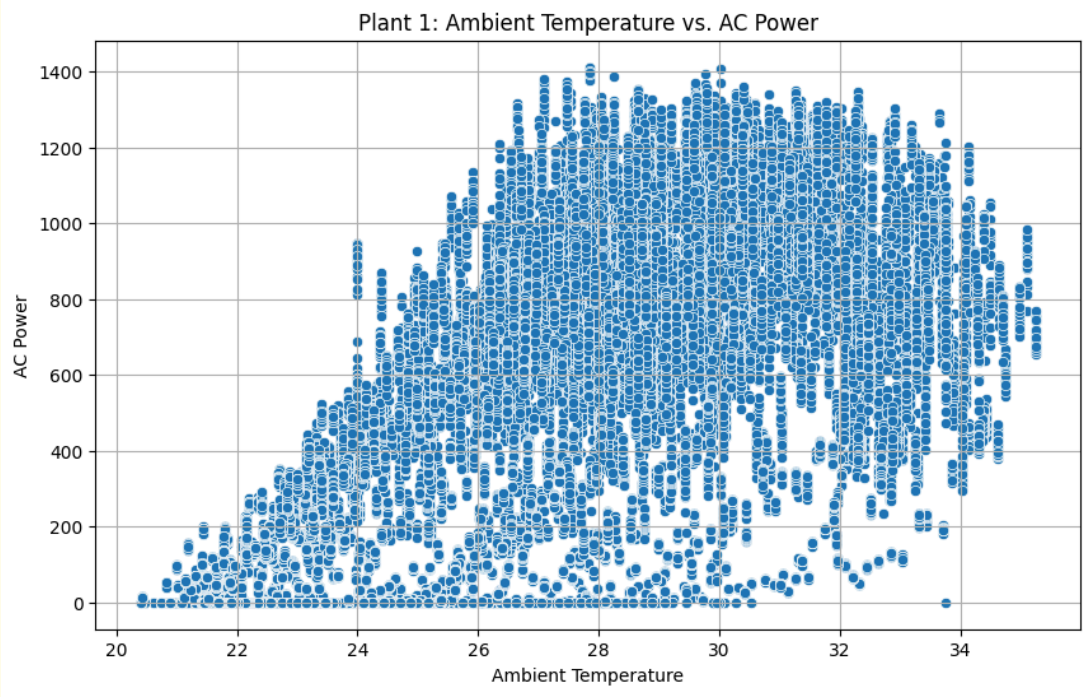
The experiment is repeated for the 2 plants to understand if the correlation holds across the data sets.

Ambient Temperature and Power Generation

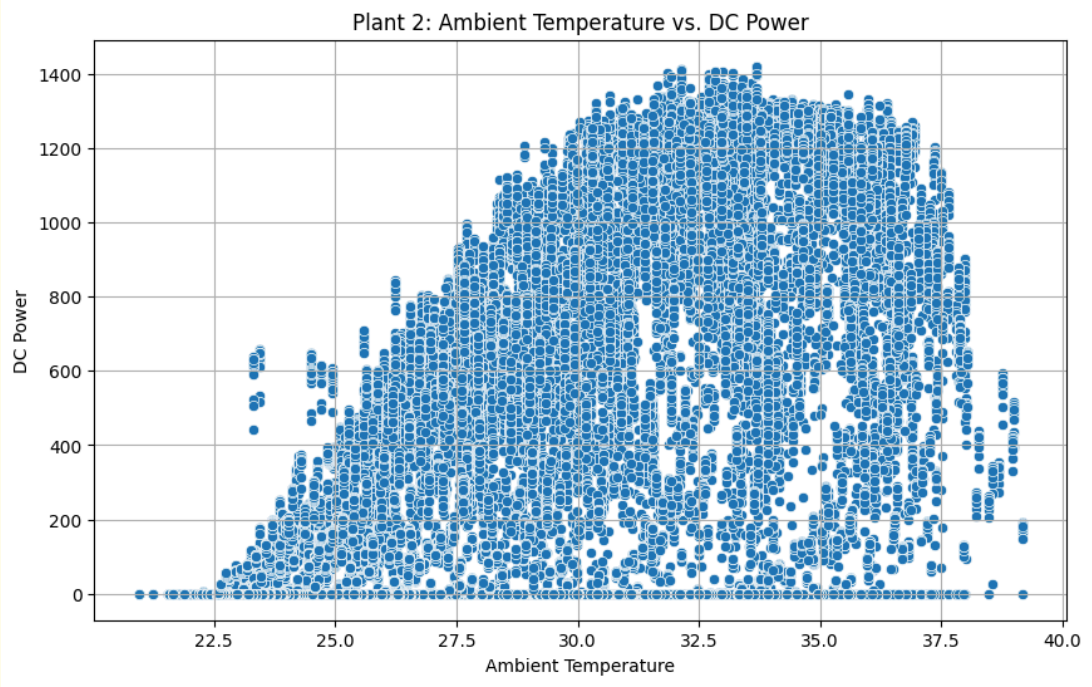
We can observe a strong positive relationship between ambient temperature and power generation. As the temperature goes up more power is being generated.

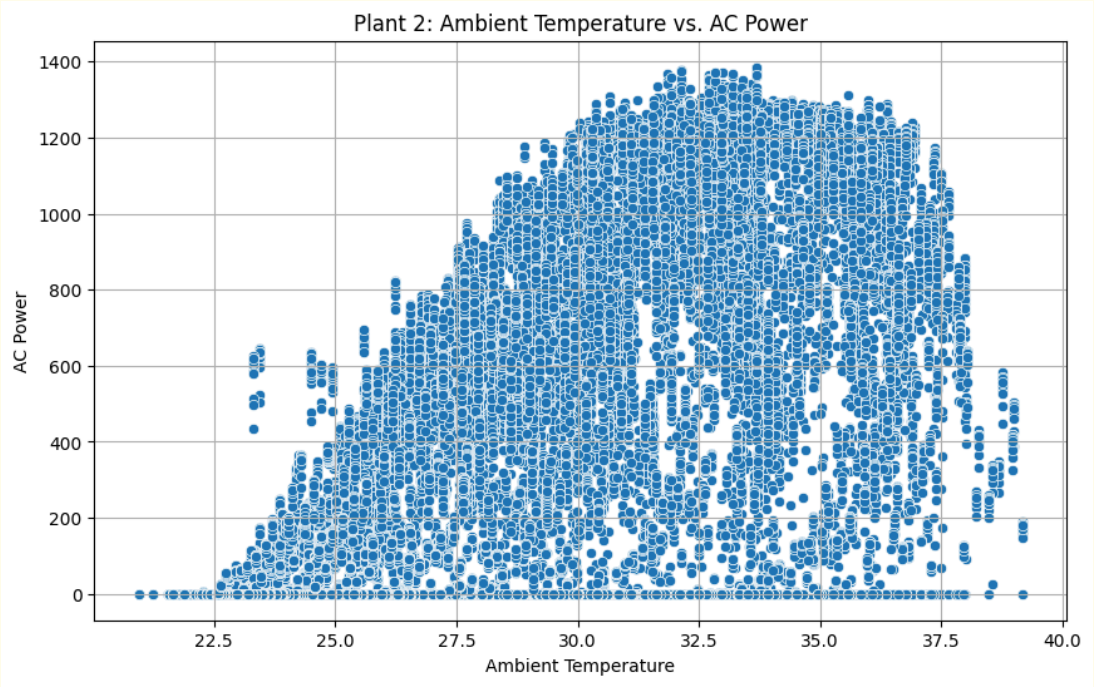
Correlation between
Ambient Temperature and DC Power for Plant 1: **0.7247**
Ambient Temperature and AC Power for Plant 1: **0.7249**





For plant 2, the correlation is weaker.
Correlation between
Ambient Temperature and DC Power for Plant 2: **0.5632**
Ambient Temperature and AC Power for Plant 2: **0.5633**





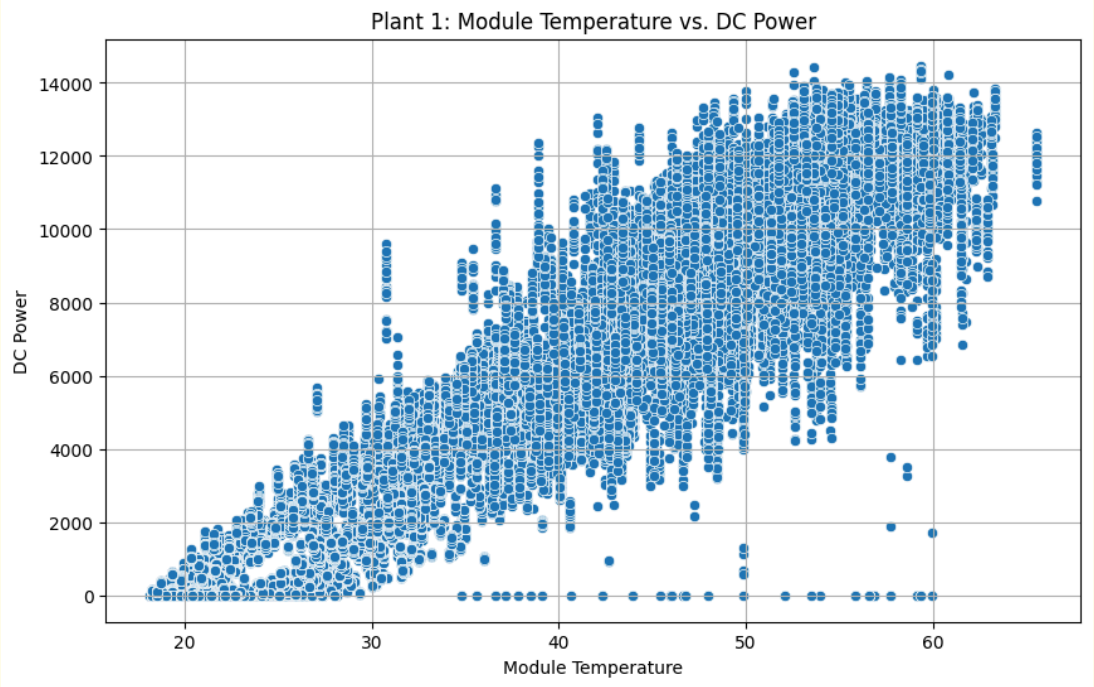
Module Temperature and Power output

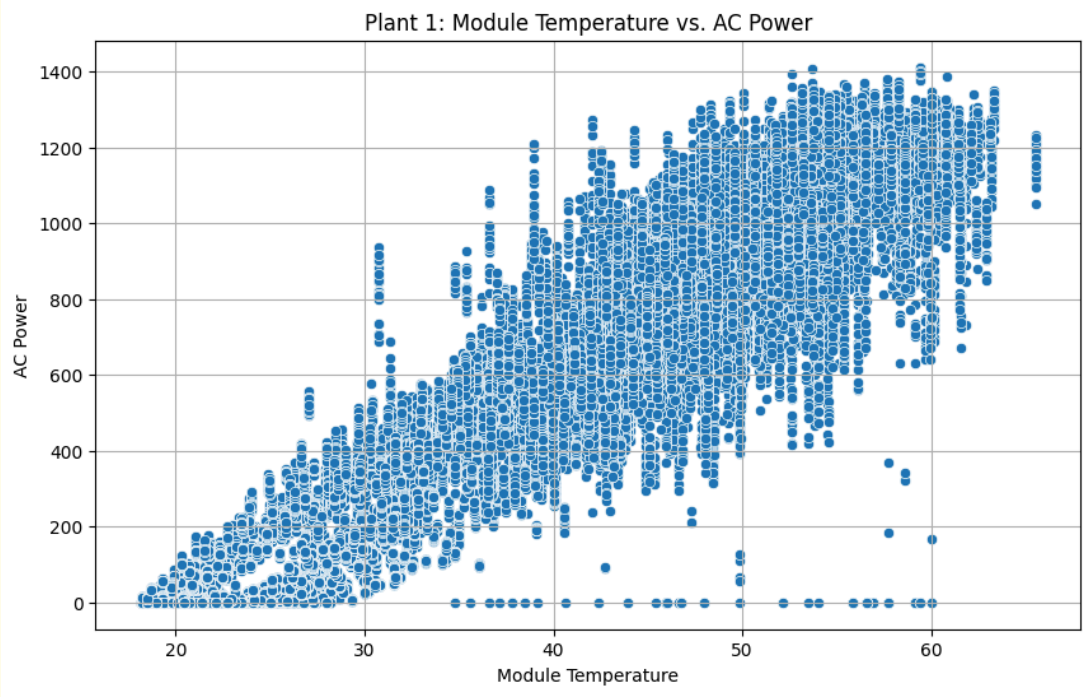
As the module temperature goes up the power output increases in a highly correlated fashion.

Correlation between

Module Temperature and DC Power for Plant 1: **0.9548**

Module Temperature and AC Power for Plant 1: **0.9549**

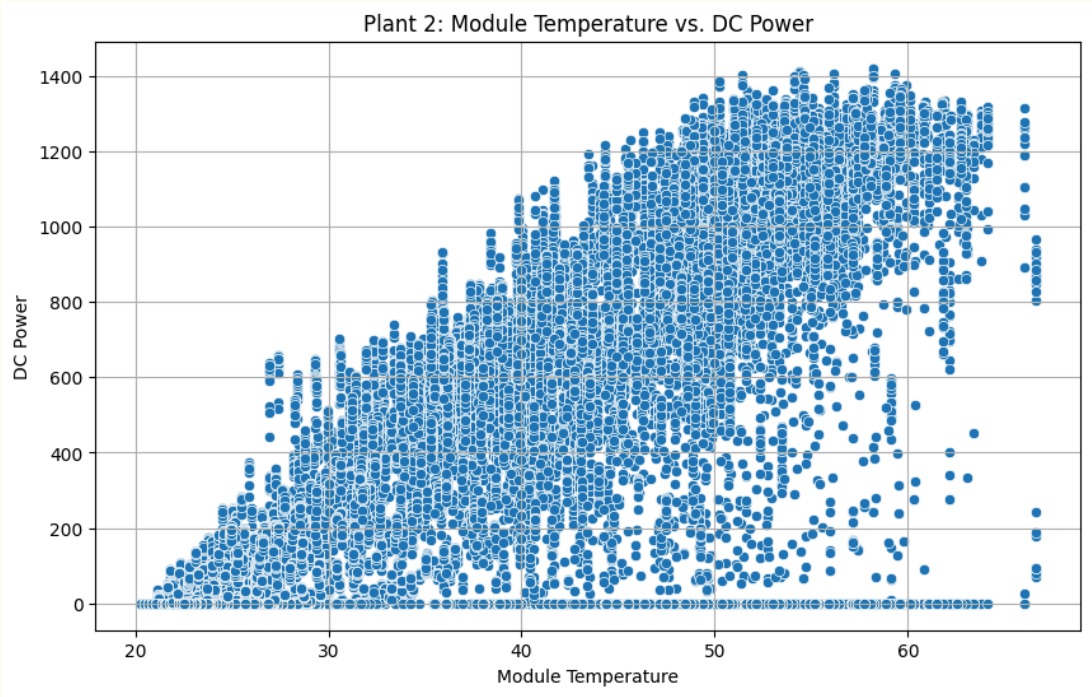


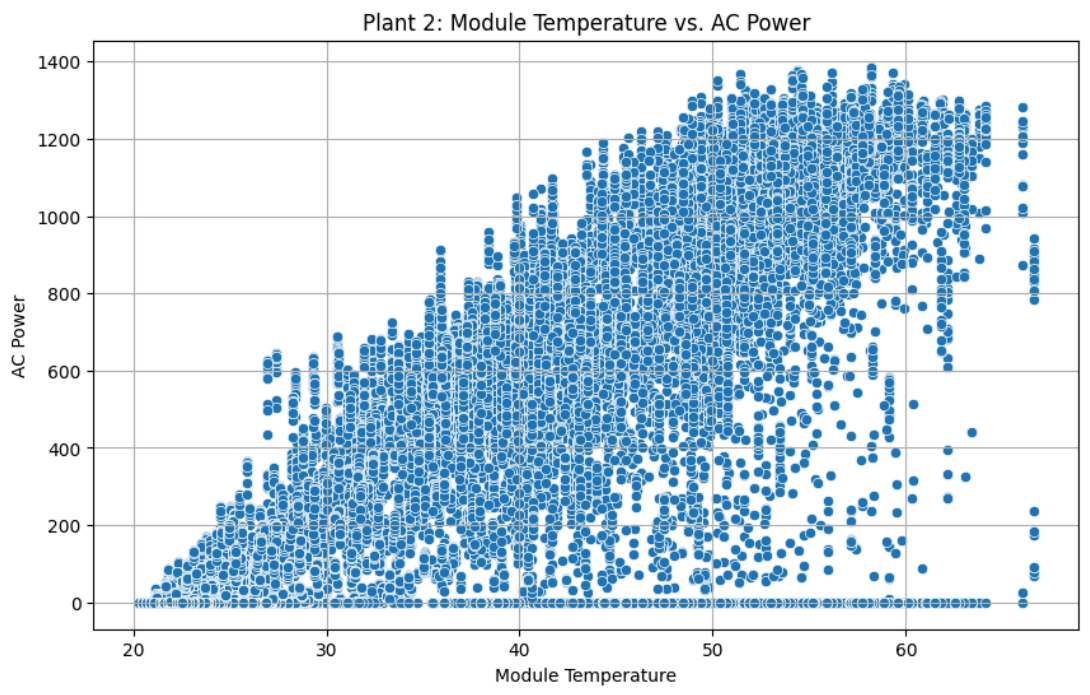


Correlation between

Module Temperature and DC Power for Plant 2: **0.7497**

Module Temperature and AC Power for Plant 2: **0.7496**



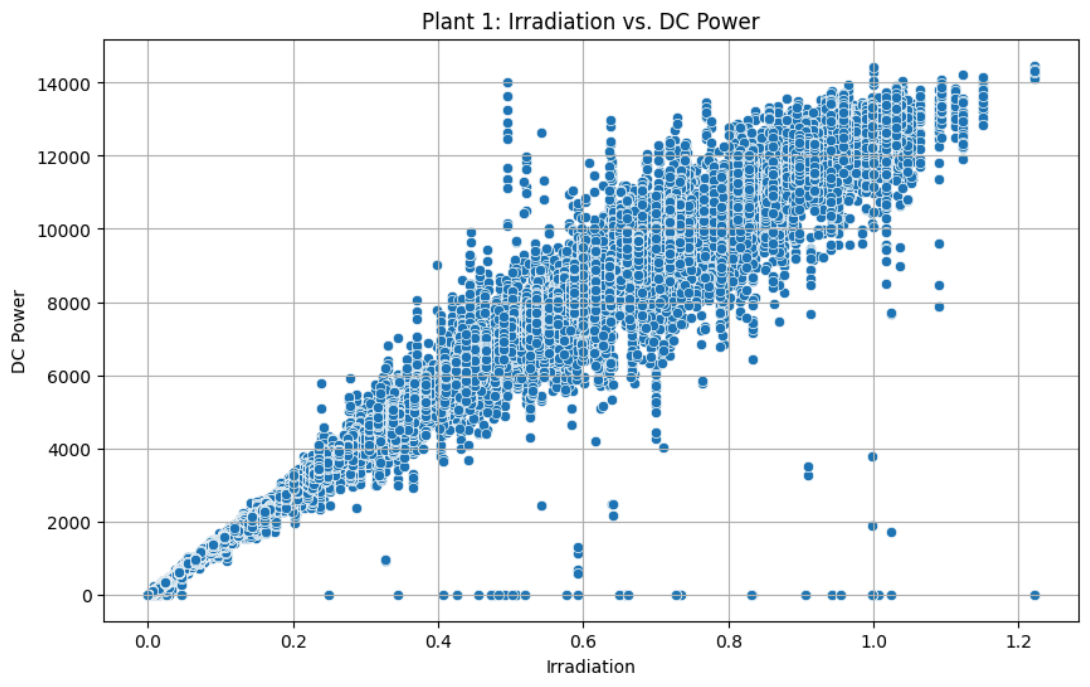


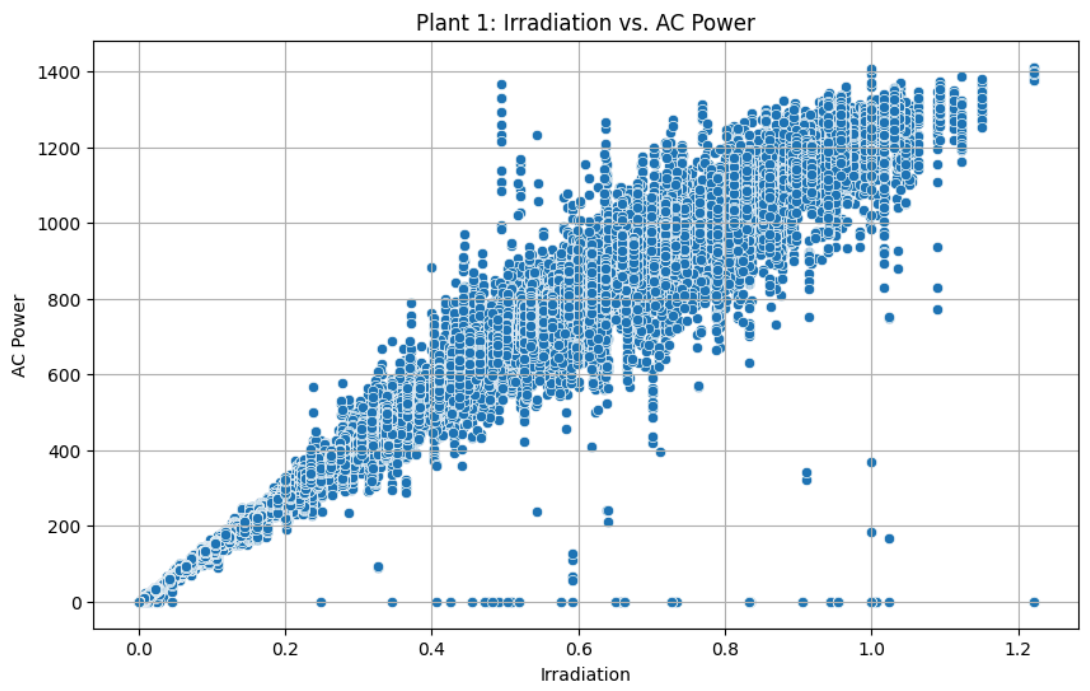
Irradiation and power output

```
    # Calculate correlations for Plant 2 (Irradiation vs Power Generation)
    corr_plant2_irradiation_dc =
merged_power_weather_plant2['IRRADIATION'].corr(merged_power_weather_plant2['DC_POWER'])
    corr_plant2_irradiation_ac =
merged_power_weather_plant2['IRRADIATION'].corr(merged_power_weather_plant2['AC_POWER'])
    print(f"Correlation between Irradiation and DC Power for Plant 2: {corr_plant2_irradiation_dc:.4f}")
    print(f"Correlation between Irradiation and AC Power for Plant 2: {corr_plant2_irradiation_ac:.4f}")

    # Plot scatter chart for Plant 2 (Irradiation vs DC Power)
    plt.figure(figsize=(10, 6))
    sns.scatterplot(x='IRRADIATION', y='DC_POWER', data=merged_power_weather_plant2)
    plt.title('Plant 2: Irradiation vs. DC Power')
    plt.xlabel('Irradiation')
    plt.ylabel('DC Power')
    plt.grid(True)
    plt.show()
```

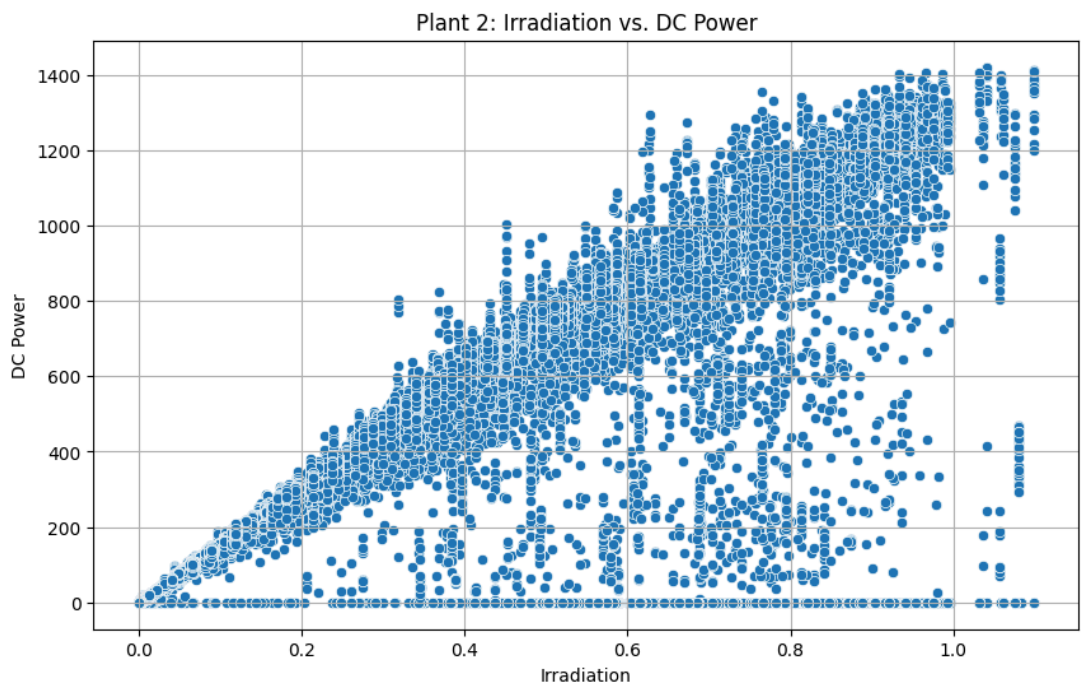
Expectedly, solar irradiation has a storing positive correlation with power generation. The existence of additional solar irradiation allows the panels to convert the solar energy to more power.

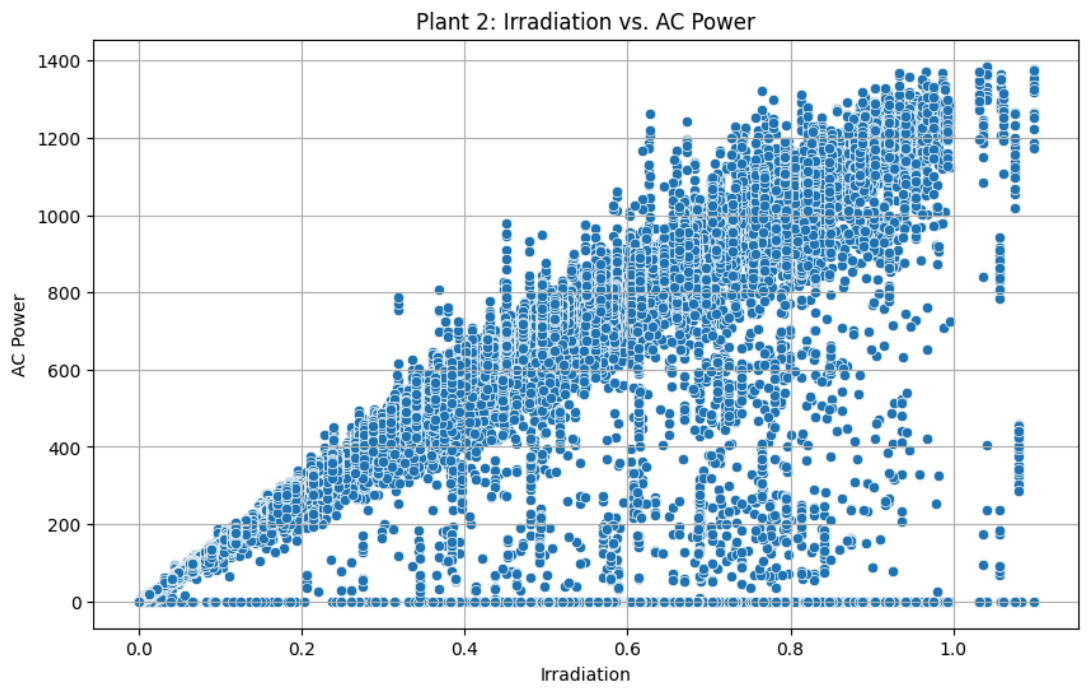
Correlation between Irradiation and DC Power for Plant 1: **0.9894**
Correlation between Irradiation and AC Power for Plant 1: **0.9893**





Correlation between Irradiation and DC Power for Plant 2: **0.7810**
Correlation between Irradiation and AC Power for Plant 2: **0.7809**


Plant 2: Irradiation vs. DC Power
DC Power
Irradiation
0
200
400
600
800
1000
1200
1400
0.0
0.2
0.4
0.6
0.8
1.0


Plant 2: Irradiation vs. AC Power
AC Power
Irradiation
0
200
400
600
800
1000
1200
1400
0.0
0.2
0.4
0.6
0.8
1.0

Cleaning Sensor Data

The sensor data is cleansed using linear interpolation when possible. The new csv files produced after cleansing the data is used for further analysis.

```
# Apply linear interpolation for numerical columns df[numeric_cols] =
df[numeric_cols].interpolate(method='linear', limit_direction='both')
# Fill any remaining NaNs (e.g., at ends if entire column was NaN) with ffill then bfill
df[numeric_cols] = df[numeric_cols].fillna(method='ffill').fillna(method='bfill')
```

Energy Efficiency

Inverter efficiency is defined as the ratio of AC power to DC power. The rows with DC power equal to zero are excluded with efficiency assumed to be 0. This could be due to low solar irradiance or other causes.

```
merged_plant2_data['INVERTER_EFFICIENCY'] = merged_plant2_data['AC_POWER'] /
merged_plant2_data['DC_POWER']
```

Energy efficiency is computed by identifying the ideal DC power from solar irradiance. The other parameters are omitted for this exercise.

IDEAL_DC_POWER = IRRADIATION × (Max DC_POWER / IRRADIATION Ratio)

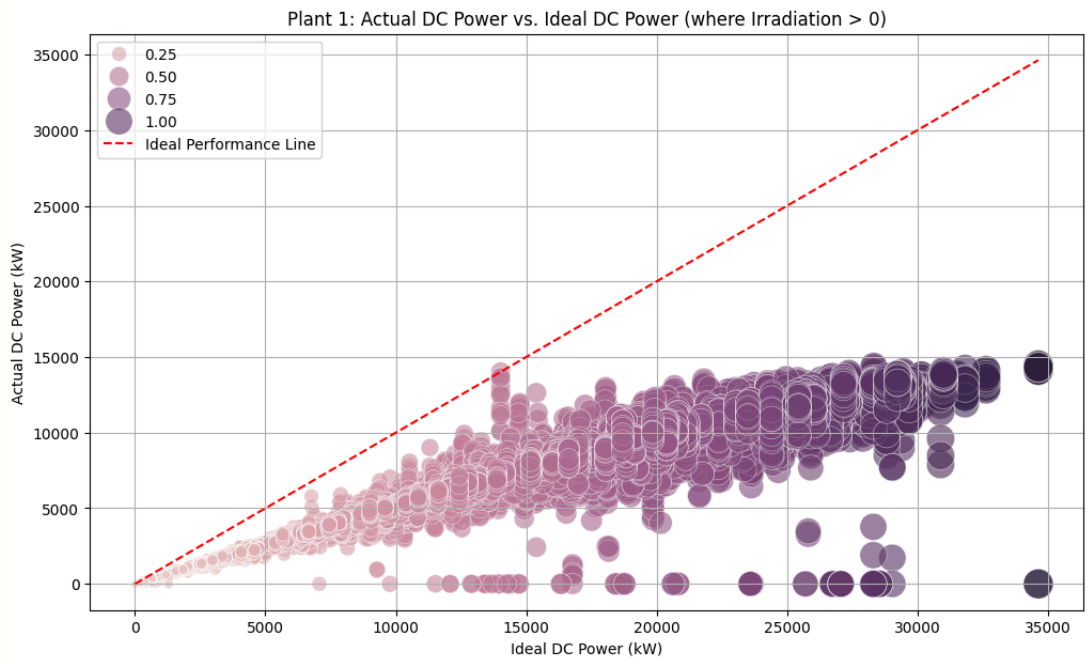
Plant1 Efficiency Metrics

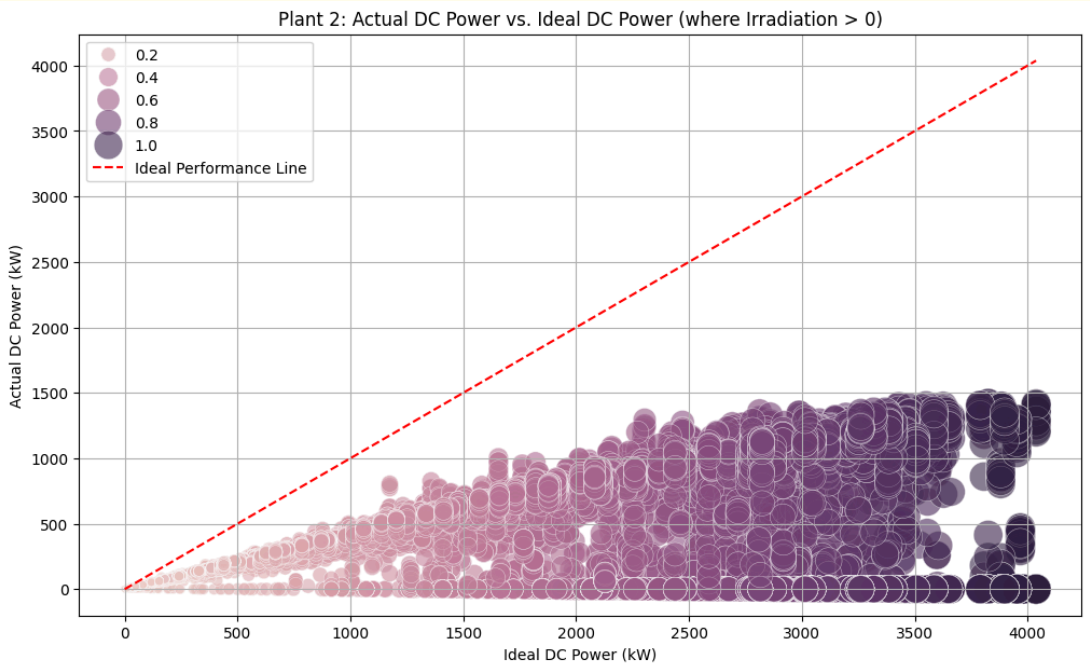
Date	Irradiation	DC_Power	Ideal_DC_Power	AC_Power	Inverter_Efficiency
6/13/2020	0.855400107	13013.57143	24250.16962	1268.814286	0.097499314
6/14/2020	1.016687181	11859.25	28822.57834	1156.6625	0.097532517
6/15/2020	1.016687181	12599.875	28822.57834	1228.8	0.097524777
6/16/2020	1.016687181	11505.71429	28822.57834	1122.842857	0.097590017
6/17/2020	1.016687181	12365.57143	28822.57834	1206.042857	0.097532319
6/18/2020	1.016687181	13485.85714	28822.57834	1314.9	0.097502145
6/19/2020	1.016687181	12505.85714	28822.57834	1219.742857	0.097533727
6/20/2020	1.016687181	9901.285714	28822.57834	966.6285714	0.097626571
6/21/2020	1.016687181	13040	28822.57834	1271.7	0.097523006
6/22/2020	1.016687181	12589.14286	28822.57834	1227.857143	0.097533022

Plant2 Efficiency Metrics

Date	Irradiance	DC_Power	Ideal_DC_Power	AC_Power	Inverter_Efficiency
6/13/2020	0.310395725	468.96	1140.634217	459.9133333	0.980709087
6/14/2020	0.310395725	451.5266667	1140.634217	442.7666667	0.980599153
6/15/2020	0.310395725	427.6533333	1140.634217	419.6266667	0.981230904
6/16/2020	0.310395725	490.8333333	1140.634217	481.1466667	0.980264856
6/17/2020	0.310395725	355.3933333	1140.634217	348.8733333	0.98165413
6/18/2020	0.310395725	425.5466667	1140.634217	417.64	0.981419977
6/19/2020	0.310395725	450.2666667	1140.634217	441.7	0.980974237
6/20/2020	0.310395725	448.78	1140.634217	440.08	0.980614109
6/21/2020	0.310395725	462.6066667	1140.634217	453.7733333	0.980905305

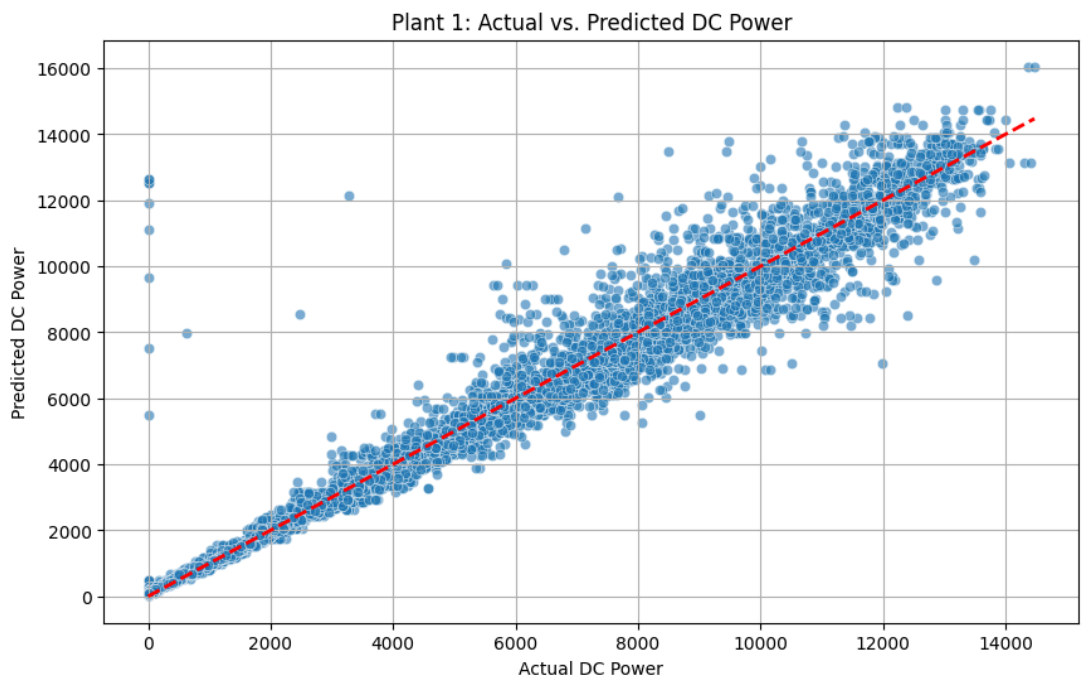
The tables above capture the data for the 2 plants. The inverter efficiency appears to be very low for plant1. In order to understand the energy efficiency, the actual and ideal DC power is plotted in the graphs below.



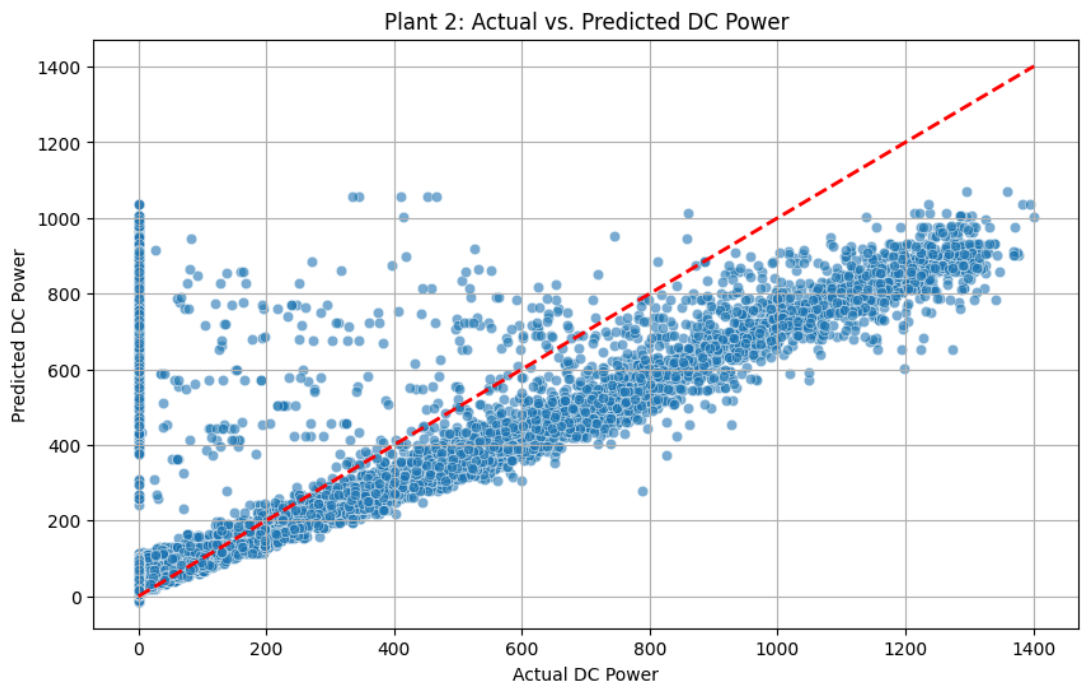


The actual DC power generation appears to significantly lag the ideal DC power for both the power plants.

A linear regression model is shown in the graph below. For Plant 1, the actual and predicted values are relatively close (determined by R-squared value of 0.98). Solar irradiation is the dominant factor. However, the ambient temperature appears to negatively impact power generation (Efficiency drops at higher temperatures).



Plant 2 shows a much larger variance with an R-squared value of 0.61. Based on the co-efficients irradiation shows the strongest positive impact.



Key Contributions

- There is a strong positive correlation between solar irradiance and power generation.
- There is a positive correlation between ambient temperature and power generation. Its effect is less pronounced than solar irradiance.
- The module temperature shows strong positive correlation which is surprising as the literature

indicates decreased efficiency in power generation with higher module temperature.

Week1

The data sets did not have data on parameters such as angle and soiling which are critical to generate a predictive model. The data from the current data sets shows no dips in power generation that would expose a negative correlation. The above data set is insufficient to predict maintenance activity. While the necessary environment for the analysis has been successfully built, further data collection is necessary to understand the impact of soiling, panel angle and weather patterns.

Week2

The data sets were cleansed and the ideal DC power output is computed. The actual DC power output for both the plants significantly lag behind the ideal output. In addition, Plant1 seems to show poor inverter efficiency as well as ideal DC power.

Week 3

Maintenance Alert Logic: Machine Learning (ML) Based Approach

To more robustly detect performance drops, especially considering factors like ambient temperature, we will use our previously trained linear regression models. These models predict an expected DC_POWER based on IRRADIATION, AMBIENT_TEMPERATURE, and MODULE_TEMPERATURE.

Our ML-based alert system will operate as follows:

1. Predict Expected DC Power: For each data point, we will use the appropriate linear regression model (model_plant1 or model_plant2) to predict the DC_POWER based on its environmental conditions.
2. Compare Actual vs. Predicted: An alert will be triggered if the actual DC_POWER falls below a certain percentage of this predicted_DC_POWER. This directly addresses the scenario where 'energy output drops for the same ambient temperature'.

This approach allows for a more nuanced and context-aware alert system than a simple rule-based threshold.

```
def generate_ml_maintenance_alerts(df, model, features,
prediction_threshold_ratio=0.85):
    """
    Generates maintenance alerts based on a machine learning model's
prediction of DC Power.

    Args:
        df (pd.DataFrame): The DataFrame containing 'DC_POWER' and the
'features' used by the model.
        model (sklearn.linear_model.LinearRegression): The trained linear
regression model.
        features (list): A list of column names in df that correspond to the
model's features.
        prediction_threshold_ratio (float): The ratio of actual DC_POWER
to predicted DC_POWER.
                                            An alert is triggered if DC_POWER <
prediction_threshold_ratio * predicted_DC_POWER.

    Returns:
        pd.DataFrame: The DataFrame with an 'ML_ALERT' column
indicating maintenance alerts.
    """
    df_ml_alerts = df.copy()
    df_ml_alerts['ML_ALERT'] = False

    # Only make predictions where IRRADIATION is positive, as DC_POWER
is 0 when no light
    active_generation_indices =
df_ml_alerts[df_ml_alerts['IRRADIATION'] > 0].index
```

```
    if not active_generation_indices.empty:
        # Prepare features for prediction
        X_predict = df_ml_alerts.loc[active_generation_indices, features]

        # Predict expected DC_POWER
        df_ml_alerts.loc[active_generation_indices,
'PREDICTED_DC_POWER'] = model.predict(X_predict)

        # Rule: Actual DC_POWER significantly lower than predicted
DC_POWER
        condition_ml_alert = ((df_ml_alerts['DC_POWER'] <
prediction_threshold_ratio * df_ml_alerts['PREDICTED_DC_POWER']) &
                              (df_ml_alerts['DC_POWER'] > 0)) # Ensure we only alert
on actual power generation drops
        df_ml_alerts.loc[condition_ml_alert, 'ML_ALERT'] = True
    else:
        df_ml_alerts['PREDICTED_DC_POWER'] = 0

    return df_ml_alerts
```

Simulating Noisy Weather Data

To test the robustness of our maintenance alert system against 'noisy' or unexpected weather fluctuations, we will create synthetic datasets by adding random noise to the environmental parameters: IRRADIATION, AMBIENT_TEMPERATURE, and MODULE_TEMPERATURE.

This simulation will involve:

1. Adding Gaussian Noise: Applying a random normal distribution to each environmental feature.
2. Recalculating Metrics: Recomputing IDEAL_DC_POWER and INVERTER_EFFICIENCY based on these noisy parameters.
3. Applying Alert Logic: Running the generate_maintenance_alerts function on the noisy data.
4. Comparison: Observing how the alert system responds to these fluctuations compared to the alerts on the original data.

```
print('\n--- Starting Noisy Weather Data Simulation ---')
```

```
def add_weather_noise(df, irradiation_noise_std=0.05,
temp_noise_std=2.0, seed=42):
    np.random.seed(seed)
    noisy_df = df.copy()

    if 'IRRADIATION' in noisy_df.columns:
        noise = np.random.normal(0, irradiation_noise_std,
size=len(noisy_df))
        noisy_df['IRRADIATION'] = noisy_df['IRRADIATION'] + noise
        noisy_df['IRRADIATION'] = noisy_df['IRRADIATION'].clip(lower=0)

    if 'AMBIENT_TEMPERATURE' in noisy_df.columns:
        noise = np.random.normal(0, temp_noise_std, size=len(noisy_df))
        noisy_df['AMBIENT_TEMPERATURE'] =
noisy_df['AMBIENT_TEMPERATURE'] + noise

    if 'MODULE_TEMPERATURE' in noisy_df.columns:
        noise = np.random.normal(0, temp_noise_std, size=len(noisy_df))
        noisy_df['MODULE_TEMPERATURE'] =
noisy_df['MODULE_TEMPERATURE'] + noise

    return noisy_df
```

Evaluate ML Alert Accuracy: Comparing Alerts from Original vs. Noisy Data

Now, let's evaluate the performance of our ML-based alert system by comparing alerts triggered on the original data versus the noisy data. This will help us understand its sensitivity to weather fluctuations and potential for false positives/negatives.

```
# Define the features used by the models
model_features = ['IRRADIATION', 'AMBIENT_TEMPERATURE',
'MODULE_TEMPERATURE']

# --- Apply ML alert logic to Plant 1 data (original) ---
print("\n--- Plant 1 ML Alerts (Original Data) ---")
ml_alerts_original_plant1 =
generate_ml_maintenance_alerts(merged_plant1_data.copy(),
model_plant1, model_features)
```

```
display(ml_alerts_original_plant1[ml_alerts_original_plant1['ML_ALERT
']].head(10))
print(f"Total ML alerts for original Plant 1:
{ml_alerts_original_plant1['ML_ALERT'].sum()}")

# --- Apply ML alert logic to Plant 2 data (original) ---
print("\n--- Plant 2 ML Alerts (Original Data) ---")
ml_alerts_original_plant2 =
generate_ml_maintenance_alerts(merged_plant2_data.copy(),
model_plant2, model_features)
display(ml_alerts_original_plant2[ml_alerts_original_plant2['ML_ALER
T']].head(10))
print(f"Total ML alerts for original Plant 2:
{ml_alerts_original_plant2['ML_ALERT'].sum()}")

# --- Apply ML alert logic to Plant 1 data (noisy) ---
print("\n--- Plant 1 ML Alerts (Noisy Data) ---")
ml_alerts_noisy_plant1 =
generate_ml_maintenance_alerts(noisy_plant1_data.copy(),
model_plant1, model_features)
display(ml_alerts_noisy_plant1[ml_alerts_noisy_plant1['ML_ALERT']].h
ead(10))
print(f"Total ML alerts for noisy Plant 1:
{ml_alerts_noisy_plant1['ML_ALERT'].sum()}")

# --- Apply ML alert logic to Plant 2 data (noisy) ---
print("\n--- Plant 2 ML Alerts (Noisy Data) ---")
ml_alerts_noisy_plant2 =
generate_ml_maintenance_alerts(noisy_plant2_data.copy(),
model_plant2, model_features)
display(ml_alerts_noisy_plant2[ml_alerts_noisy_plant2['ML_ALERT']].
head(10))
print(f"Total ML alerts for noisy Plant 2:
{ml_alerts_noisy_plant2['ML_ALERT'].sum()}")



--- Evaluating ML Alert Accuracy ---

--- Plant 1 ML Alert Comparison ---
Total ML alerts on original Plant 1 data: 3831
Total ML alerts on noisy Plant 1 data: 3795
```

Common ML alerts (triggered in both original and noisy) for Plant 1: 45752
ML alerts only in noisy data for Plant 1 (potential noise-induced false positives): 489
ML alerts only in original data for Plant 1 (potential noise-masked true positives): 15

--- Plant 2 ML Alert Comparison ---
Total ML alerts on original Plant 2 data: 4770
Total ML alerts on noisy Plant 2 data: 4648
Common ML alerts (triggered in both original and noisy) for Plant 2: 76026
ML alerts only in noisy data for Plant 2 (potential noise-induced false positives): 70
ML alerts only in original data for Plant 2 (potential noise-masked true positives): 5

Interpretation of ML-Based Alert Accuracy Results

This comparison provides insights into how the ML-based alert system responds to both normal and noisy conditions:

- Robustness: A high proportion of Common ML alerts relative to the total alerts suggests good robustness, meaning the system can identify underlying issues even with some weather fluctuations.
- False Positives: ML alerts only in noisy data indicates instances where noise alone was enough to trigger an alert. This might suggest the prediction_threshold_ratio is too low, or that the model is sensitive to the specific type of noise introduced.
- False Negatives: ML alerts only in original data could mean that the noise inadvertently pushed the DC_POWER back into an 'acceptable' range relative to the prediction, masking a genuine issue present in the original data.

Compared to simple rule-based systems, ML models inherently factor in the complex interplay of environmental variables (like ambient temperature, module temperature, and irradiation) to determine what constitutes a 'normal' expected output. Deviations from this predicted normal then trigger alerts.

Week 4

The above approach was replaced with the generation of synthetic data where artificial drops in energy output are injected at specific time frames. These time frames are captured so that the predicted results could be compared with the expected. A visual representation of the actual and expected results is far easier to consume and debug. The following additional changes were made

- Restricted the model to use just the data from plant 1.
- Moved away from introducing noise in the data to synthetic data generation.
- The false positives found multiple bugs in the code. Merging the tables had a bug. When solar radiation is low (in the morning) the power output reduction is non-linear. Added a higher threshold to reduce false positives
- Significant code simplification was done.

Conclusions

Week1
The data sets did not have data on parameters such as angle and soiling which are critical to generate a predictive model. The data from the current data sets shows no dips in power generation that would expose a negative correlation. The above data set is insufficient to predict maintenance activity. While the necessary environment for the analysis has been successfully built, further data collection is necessary to understand the impact of soiling, panel angle and weather patterns.

Week2
The data sets were cleansed and the ideal DC power output is computed. The actual DC power output for both the plants significantly lag behind the ideal output. In addition, Plant1 seems to show poor inverter efficiency as well as ideal DC power.

Week 3
Added the simple rule based model first
Modified it to use Linear regression model
Created Alert generation logic
Introduced noise to test the logic

Week 4
Restrict the model to use just the data from plant 1.
Moved away from introducing noise in the data to synthetic data generation.
The false positives found multiple bugs in the code. Merging the tables had a bug. When solar radiation is low (in the morning) the power output reduction is non-linear. Added a higher threshold to reduce false positives
Significant code simplification was done.

References

[1] Kutner, M. H., Nachtsheim, C. J., Neter, J., & Li, W. (2004). *Applied linear statistical models* (5th ed.). McGraw-Hill Irwin.
[2] **Mertens, K. (2018).** *Photovoltaics: Fundamentals, technology, and practice* (2nd ed.)